데이터과학 파이썬

심송용(한림대학교 데이터과학스쿨)

http://jupiter.hallym.ac.kr

자료구조

defaultdict: 키(key)값이 없을 때 반환할 값을 지정할 수 있는 딕셔너리이다. key값이 없을 때 사용할 함수를 매개변수로 사용하여 defaultdict를 생성한다. 예를 들어 key값이 없을 때 undef_key 라는 함수를 호출하는 defaultdict는 d = defaultdict(undef_key)와 같이 사용한다.

```
# collections3.py
import collections as cln

def undefined_key():
    return "해당 자료없음"

# Defining the dict
d = cln.defaultdict(undefined_key)
d["a"] = 1; d["b"] = 2

print(d["a"], d["b"], d["c"])
```

자료구조

1 2 해당 자료없음
로 key 값 c에 해당하는 자료가 없을 때 undefined_key를 호출한 결과를 반환한다.

Counter: 시퀀스 형태의 자료에서 자료값의 빈도를 딕셔너리 형태로 반환한다. 도수분포표, 텍스트 마이닝 등에서 유용하다. 빈도수의 크기에 따라 정렬된 출력을 얻는다.
다음은 특정 단어에서 사용한 문자의 빈도를 계산한 것이다.

```
# collections4.py
import collections as cln

mytext = 'collections'
lmytext = list(mytext)
print(lmytext)
freq = cln.Counter(lmytext)
print(freq)
```

자료구조

['c', 'o', 'l', 'l', 'e', 'c', 't', 'i', 'o', 'n', 's']
Counter({'c': 2, 'o': 2, 'l': 2, 'e': 1, 't': 1, 'i': 1, 'n': 1, 's': 1})

출력결과는 빈도수의 값에 따라 정렬되어 있음을 알 수 있다.

앞에서 본 BTS의 Dynamite 가사에서 각 단어별 빈도를 얻어보자. # collections4.py

```
import collections as cln

f = open(r'D:\HWP\강의자료\파이썬\codes\BTS_Dynamite_lyrics.txt', 'r')
lyrics = f.readlines() # 한 번에 다 읽음
f.close()

mywords = ''.join(lyrics) # lyrics 가 리스트이므로 하나의 문자열로 변경

freq=cln.Counter(mywords.lower().split())
print(freq)
```

자료구조

Counter({'the': 33, "i'm": 14, 'so': 14, 'and': 14, 'up': 13, 'like': 13, 'na-na,': 12, 'i,': 10, 'is': 10, 'dyn-na-na-na,': 10, 'a': 9, 'it': 9, 'light': 8, 'dynamite': 7, 'in': 6, 'me': 6, 'bring': 6, 'life': 6, 'jungkook,': 6, 'shining': 6, 'through': 6, 'city': 6, 'with': 6, 'little': 6, 'funk': 6, 'soul': 6, "i'ma": 6, 'ayy': 6, "'cause": 5, 'stars': 5, 'tonight': 5, 'watch': 5, 'fire': 5, 'set': 5, 'night': 5, 'alight': 5, 'you': 5, 'dynamite,': 5 ...

이 경우 콤마, 괄호 등 특수한 문자를 고려하지 않았음에 유의.

Counter 개체의 생성은 딕셔너리 형태 및 key=freq 형태로 생성할 수 있으며, 빈도수에 대한 연산도 가능하다.

```
cc = cln.Counter({'red':4,  'blue':3})
print("cc: ", list(cc.elements()))
```

자료구조

```
dd = cln.Counter(red=3, blue=4)
print("dd: ", list(dd.elements()))

ee = dd + cc
print("dd+cc: ", dd + cc)
print("dd-cc: ", dd - cc)
ee.subtract(dd)
print(ee)
```

```
cc:  ['red', 'red', 'red', 'red', 'blue', 'blue', 'blue']
dd:  ['red', 'red', 'red', 'blue', 'blue', 'blue', 'blue']
dd+cc:  Counter({'red': 7, 'blue': 7})
dd-cc:  Counter({'blue': 1})
Counter({'red': 4, 'blue': 3})
```

자료구조

namedtuple: 간편한 방법으로 개체를 생성하는 모듈이다. 예를 들어 3차원 공간에서 좌표를 저장하는 점을 정의하고자 하면

Point = namedtuple('Point', 'x, y z')

과 같이 할 수 있으며 이 때 생성된 개체의 속성은 x, y, z 세 개가 된다. 이 속성은 namedtuple 이름 뒤에 점(.)을 사용하여 .x, .y, .z 로 속성의 값을 부를 수 있다. 다만 근본이 tuple이라 속성의 값을 바꿀 수는 없다.

```
# collections5.py
import collections as cln

def dist3D(p, q):
  distance =  (p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2
  return(distance ** 0.5)

Point = cln.namedtuple('Point', 'x, y z')

p1 = Point(0,0,0)
```

자료구조

```
p2 = Point(1,1,1)
print(dist3D(p1,p2))

Person = cln.namedtuple('Person', ['name', 'age', 'gender'])
pp1 = Person('김서은', 24, 'F')
print(pp1)
print("이름은 ", pp1.name, " 나이는 ", pp1.age, " 성별은 ", pp1.gender)
```

```
1.7320508075688772
Person(name='김서은', age=24, gender='F')
이름은  김서은  나이는  24  성별은  F
```

자료구조

LAB: Text Mining

문장에서 단어의 빈도수 확인

- defaultdict를 사용하여 빈도수 저장
- 빈도수에 따라 내림차순 정렬한다.
- OrderedDict를 사용한다.

```
# collections6.py
import collections as cln

f = open(r'D:\HWP\강의자료\파이썬\codes\BTS_Dynamite_lyrics.txt', 'r')

w_counts = cln.defaultdict(lambda: 0)

while True:
    lyrics = f.readline()
    if not lyrics: break
```

자료구조

```
    for word in lyrics.lower().split():
        w_counts[word] += 1
f.close()

for k, v in cln.OrderedDict(sorted(w_counts.items(),
                    key=lambda t: t[1], reverse=True)).items():
    print(k, v)
```

the 33	i'm 14	so 14	and 14	up 13
like 13	na-na, 12	i, 10	is 10	dyn-na-na-na, 10
a 9	it 9	light 8	dynamite 7	in 6
me 6	bring 6	life 6	jungkook, 6	shining 6
through 6	city 6	with 6	little 6	funk 6
soul 6	i'ma 6	ayy 6	'cause 5	stars 5
tonight 5	watch 5	fire 5	set 5	night 5
alight 5	you 5	dynamite, 5		

자료구조

참고: lambda 함수

lambda 함수는 작은 실행을 하기 위한 이름이 없는(anonymous) 함수이다. 사용법은

```
lambda arguments : expression
```

이며 아주 단순하게

lambda: 0

이면 매개변수 없이 호출되어 0이 반환된다. 위의 코드에서 사용한

lamda t: t[1]

은 t를 매개변수로 받아 t의 두 번째 값이 반환된다. 따라서

sorted(w_counts.items(), key=lambda t: t[1], reverse=True)

는 별도의 함수와 한 개의 명령

```
def sort_by_value(t):
    return t[1]
```

sorted(w_counts.items(), key=sort_by_value, reverse=True)

를 대신할 수 있는 편한 방법이다.

수고하셨습니다^^